

SORTING ALGORITHMS

NIIVEDHITHA . G

192521231

CSA0313

DATA STRUCTURE

1. Insertion Sort

Idea : Builds the sorted list one element at a time by inserting each element into its correct position.

Steps :

1. Start from the second element.
2. Compare it with previous element.
3. Shift larger elements to the right.
4. Insert the element in the correct position.

Time : Best $O(n)$, Average $O(n^2)$, Worst $O(n^2)$; Space : $O(1)$.

2. Selection Sort

Idea : Repeatedly finds the smallest element and places it at the beginning.

Steps :

1. Find the minimum element.
2. Swap it with the first unsorted element.
3. Move to the next position.
4. Repeat until sorted.

Time : Best $O(n^2)$, Average $O(n^2)$, Worst $O(n^2)$

Space : $O(1)$

3. Merge Sort

Idea: Divides the array into smaller parts, sorts them, and merges them.

Steps:

1. Divide the array into two halves.

2. Recursively sort both halves.

3. Merge the sorted halves.

4. Continue until the complete array is sorted.

Time: Best $O(n \log n)$, Average $O(n \log n)$, Worst $O(n \log n)$

Space: $O(n)$

4. Bubble Sort

Idea: Repeatedly compares adjacent elements and swaps them if they are in the wrong order.

Steps:

1. Compare adjacent elements.

2. Swap if the first is greater than the second.

3. Continue through the array.

4. Repeat until no swaps are needed.

Time: Best $O(n)$, Average $O(n^2)$, Worst $O(n^2)$

Space: $O(1)$

5. Radix Sort

Idea: Sort numbers digit by digit, starting from the least significant digit.

Steps:

1. Find the maximum number of digits.
2. Sort elements according to each digit.
3. Use a stable sorting method such as Counting Sort.
4. Repeat for all digit positions.

Time: $O(d \times (n+k))$

Space: $O(n+k)$

Where d = no. of digits, k = range of digits.

6. Quick Sort

Idea: Selects a pivot and partitions the array around the pivot.

Steps:

1. Select a pivot.
2. Place smaller elements before the pivot.
3. Place larger elements after the pivot.
4. Recursively sort both partitions.

Time: Best $O(n \log n)$, Average $O(n \log n)$, Worst $O(n^2)$

Space: Average $O(\log n)$, Worst $O(n)$.

7. Heap Sort

Idea: Uses a binary heap to repeatedly extract the largest/smallest elements.

Steps:

1. Build a max heap.
2. Swap the root with the last element.
3. Reduce heap size.
4. Heapify the remaining elements.
5. Repeat until sorted.

Time: Best $O(n \log n)$, Average $O(n \log n)$, Worst $O(n \log n)$

Space: $O(1)$

Comparative Table:

Sorting Algorithm	Best Time	Average Time
Insertion Sort	$O(n^2)$	$O(n^2)$
Selection Sort	$O(n^2)$	$O(n^2)$
Bubble Sort	$O(n^2)$	$O(n^2)$
Merge Sort	$O(n \log n)$	$O(n \log n)$
Quick Sort	$O(n \log n)$	$O(n \log n)$
Heap Sort	$O(n \log n)$	$O(n \log n)$
Radix Sort	$O(d(n+k))$	$O(d(n+k))$